— *Solution notes* —

COMPUTER SCIENCE TRIPOS Part IB – 2022 – Paper 5

5 Concurrent and Distributed Systems (mk428)

You are designing *Warbler*, a social networking service in which users can post public messages called *chirps*. A chirp may or may not be a reply to another chirp. Moreover, one user can *follow* another user, which means subscribing to the messages they post.

replication, quorums, read-after-write consistency, ABD algorithm

(*a*) Write pseudocode for three operations: (1) user u_1 following user u_2; (2) u_1 unfollowing (ceasing to follow) u_2; and (3) obtaining the current set of followers of some user u. These operations must be fault-tolerant: you should store the data on five servers, and the operations must be able to complete successfully as long as any three or more servers are available. Your algorithm should ensure read-after-write consistency.

Assume an asynchronous system model with fair-loss network links and crash-recovery faults. Write your algorithm in terms of messages sent over point-to-point (unicast) links. You may assume that the user is already authenticated (e.g. their password has been checked), and you may assume that users' client software can generate Lamport timestamps.

Start by describing the nodes in the system, the structure of the data stored at each node, and the form and purpose of messages exchanged by nodes. Then give the pseudocode for the operations listed above. [12 marks]

Answer: We treat each user as an independent node, and each of the five servers as a node. The only data stored by users is a unique user ID and a counter for the purpose of generating Lamport timestamps. The servers store a set of 4-tuples of the form (u_1, u_2, s, t), where $s = \mathsf{true}$ if u_1 is following u_2, $s = \mathsf{false}$ if u_1 is not following u_2, and t is the timestamp of the most recent operation that updated the follow status between u_1 and u_2.

The system uses the following message types:

$(\mathsf{follow}, u_1, u_2, s, t)$	user to server: request by user u_1 to follow ($s = \mathsf{true}$) or unfollow ($s = \mathsf{false}$) user u_2; t is the Lamport timestamp of the operation, generated by the user
(ok, t)	server to user: completed follow request with timestamp t
$(\mathsf{getFollowers}, u, id)$	user to server: get followers of user u; id is a unique request identifier for associating responses with requests
$(\mathsf{followers}, response, id)$	server to user: response to request id. $response$ is a set of (u, s, t) triples where u is a follower ($s = \mathsf{true}$) or non-follower ($s = \mathsf{false}$) and the relationship was last updated by timestamp t.

A user sends each follow or getFollowers message to all of the servers, and waits for responses from a majority quorum. The timestamps need to be unique and consistent with causality: successive requests by the same user must have monotonically increasing timestamps.

When u_1 unfollows u_2, it is not sufficient to simply delete the tuple for (u_1, u_2), because if we did that we could not tell the difference between a server that has already processed the unfollow operation and a server that has not yet processed the earlier follow operation. Given that the asynchronous network can arbitrarily delay and reorder messages, unfollowing requires an update with $s = \mathsf{false}$ and a greater timestamp.

An algorithm based on consensus/total order broadcast cannot be used because such an algorithm does not exist in an asynchronous system model (as per the FLP result).

on request by u_1 to follow/unfollow u_2 **do**
  $s :=$ true **if** request is to follow
  $s :=$ false **if** request is to unfollow
  $t :=$ fresh logical timestamp (e.g. Lamport timestamp)
  send $(\mathsf{follow}, u_1, u_2, s, t)$ to all 5 servers
    (periodically resend message to servers that have not yet responded)
  wait for responses that look like (ok, t)
  return when 3 or more servers have responded
end on

on request to get followers of user u **do**
  $id :=$ generateUniqueID() (to associate responses with the correct request)
  send $(\mathsf{getFollowers}, u, id)$ to all 5 servers
    (periodically resend message to servers that have not yet responded)
  $responses := \{\}$; $serversResponded := \{\}$
  on receiving response $(\mathsf{followers}, response, id)$ from server n **do**
    $responses := responses \cup response$
    $serversResponded := serversResponded \cup \{n\}$
    if $|serversResponded| \geq 3$ **then**
      $maxTS := \{(u', s') \mid \exists t'.\ (u', s', t') \in responses \wedge$
                $(\nexists s'', t''.\ (u', s'', t'') \in responses \wedge t' < t'')\}$
      return $\{u' \mid (u', \mathsf{true}) \in maxTS\}$
    end if
  end on
end on

At each server:

on initialisation **do**
  $follows := \{\}$ (in persistent storage)
end on

on server receiving $(\mathsf{follow}, u_1, u_2, s, t)$ from client c **do**
  if $\exists s', t'.\ (u_1, u_2, s', t') \in follows$ **then**
    if $t' < t$ **then**
      $follows := follows \setminus \{(u_1, u_2, s', t')\} \cup \{(u_1, u_2, s, t)\}$
    end if
  else
    $follows := follows \cup \{(u_1, u_2, s, t)\}$
  end if
  send (ok, t) to client c
end on

on server receiving $(\mathsf{getFollowers}, u, id)$ from client c **do**
  $response := \{(u', s', t') \mid (u', u, s', t') \in follows\}$
  send $(\mathsf{followers}, response, id)$ to client c
end on

This algorithm relies on clients retrying their writes to the servers indefinitely, so a server that is unavailable for a while should eventually receive the writes that it missed. A more practical algorithm would recognise that clients may crash without recovering, leaving them unable to retry a follow/unfollow request. This would necessitate an additional anti-entropy protocol between servers, through which a server can learn about writes that it missed while it was offline.

These solution notes provide a lot of detail to facilitate future use of this question as an

exercise. In an exam setting, this level of detail is not expected. Full credit will be awarded for providing the key elements of a working solution and demonstrating an understanding of the key issues surrounding distributed systems.

---

causal broadcast, FIFO broadcast, logical time

(*b*) When a user posts a chirp, it needs to be sent to all of that user's followers. Write pseudocode to do this, using the operation for getting a user's followers from part (*a*). Chirps must be delivered to a given recipient in the following order: chirps posted by the same user should be delivered in the order they were posted; and when one chirp is a reply to another, the reply should be delivered after the chirp it is replying to. Apart from these rules, the chirps may be delivered in any order. [8 marks]

Answer: One possible approach would be to use a causal broadcast protocol based on vector clocks, as shown in lectures. However, in this particular situation we can actually use a simpler and more efficient protocol: we sequentially number chirps per user (to get FIFO broadcast per user), and the only causal dependency we need to track between different users is the explicit in-reply-to relationship, when present. A user receiving a chirp then delays the delivery until those dependencies are satisfied.

At each user:

on initialisation **do**
  $sendSeq := 0$; $chirps := \{\}$; $deliveredSeq := \{\}$; $deliveredIDs := \{\}$
  (all in persistent storage)
end on

on request by user u to post chirp m in reply to chirp with ID r **do**
    ($r = \mathsf{null}$ if the chirp is not in reply to anything)
  $id :=$ generateUniqueID()
  $followers :=$ get followers for user u as per part (*a*)
  send $(\mathsf{chirp}, m, r, sendSeq, id)$ to each user in $followers$
    (periodically resend message to users who have not yet responded)
  $sendSeq := sendSeq + 1$
  return id
end on

on user u_1 receiving $(\mathsf{chirp}, m, r, seq, id)$ from user u_2 **do**
  $chirps := chirps \cup \{(u_2, m, r, seq, id)\}$
  send $(\mathsf{chirpOk}, id)$ to user u_2
  while $\exists (u', m', r', seq', id') \in chirps.\ (seq' = 0 \vee (u', seq') \in deliveredSeq) \wedge$
                $(r' = \mathsf{null} \vee r' \in deliveredIDs)$
    deliver (u', m', r', id') to the application
    $deliveredSeq := deliveredSeq \setminus \{(u', seq')\} \cup \{(u', seq' + 1)\}$
    $deliveredIDs := deliveredIDs \cup \{id'\}$
  end while
end on

This algorithm leaves unclear what exactly a "user" is. Is it an app running on an end-user device such as a phone? Or is the user actually represented by a server, which runs this algorithm on the user's behalf, and serves the user their latest chirps when the user connects? If so, we would probably want to replicate the user's "inbox" of chirps. These things would be important to think through in a real implementation, but for the purposes of this exam question it is not necessary to go into so much detail. As with part (*a*), credit is awarded for showing the key elements of a working solution.

— *Solution notes* —

In both parts, in addition to the pseudocode, please also briefly explain and justify the design of your algorithm, and specify any assumptions you are making.